Start with the Goal:

The objective of this lab was to create a GitOps-based development environment where deployments happen through Git commits instead of manually running kubectl commands.

Infrastructure:

1.Git Installed

2.WSL Installed

3.Ubuntu Installed

4.Docker Installed

5.Terraform Installed

6.Kubectl Installed

7.Helm Installed

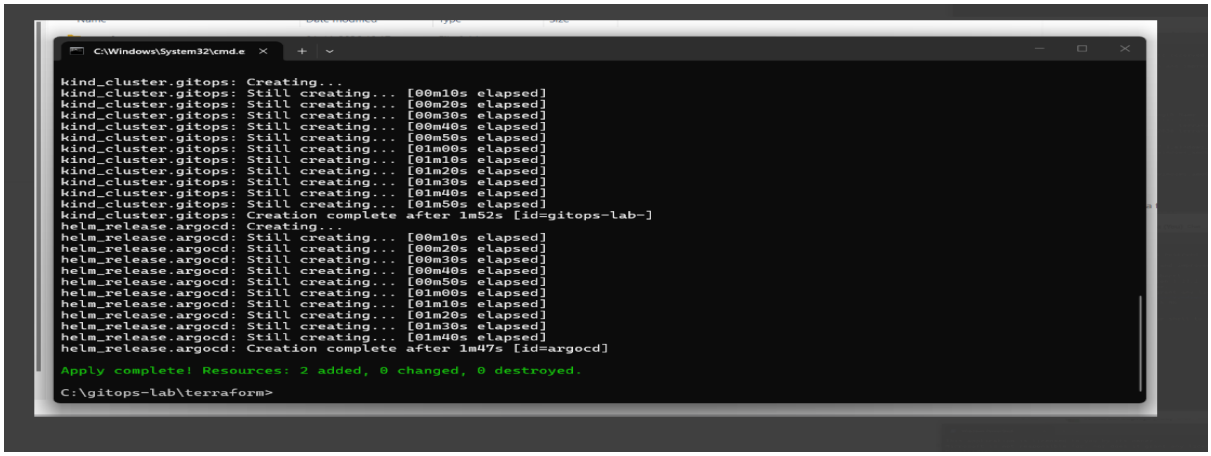
8.Kind Installed

9. GitHub Repository

10.ArgoCD

11.Gitops Lab

* I used Terraform to automatically create a Kind Kubernetes cluster and install ArgoCD using a Helm chart.



* Terraform plan confirms the infrastructure matches the desired configuration

Architecture

GitHub Repository

↓

ArgoCD

↓

Kubernetes Cluster

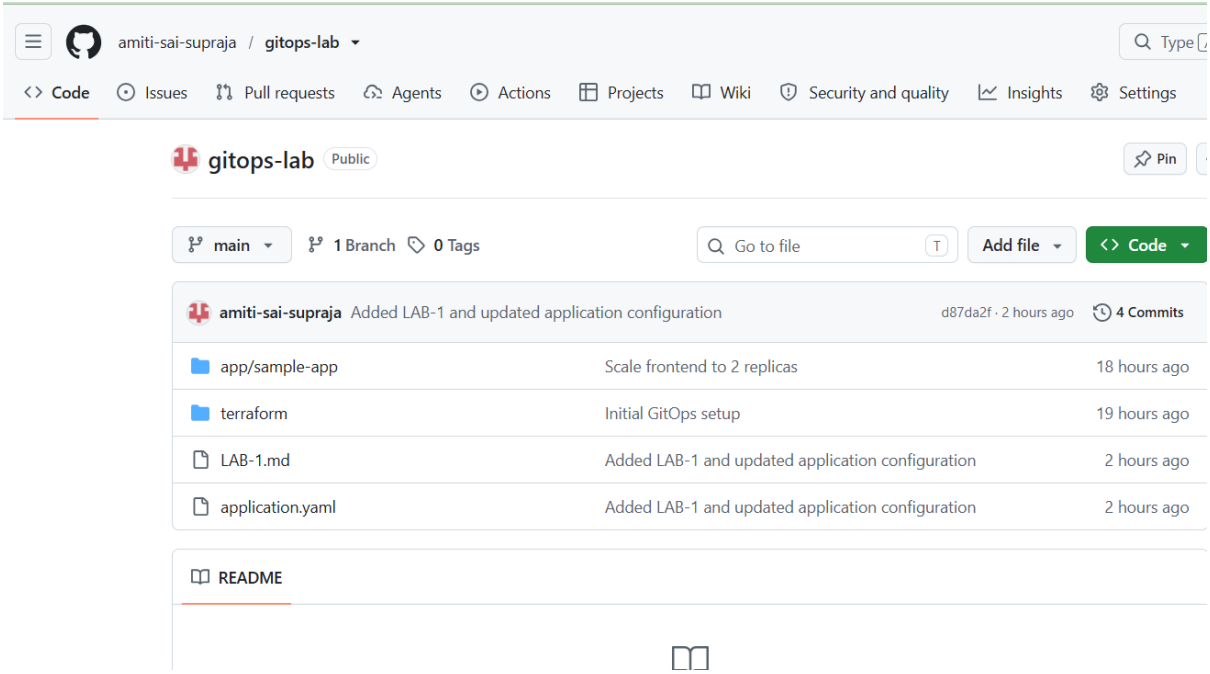
↓

sample-app

* GitHub stores the application configuration. ArgoCD continuously monitors GitHub. Whenever Git changes, ArgoCD updates Kubernetes automatically.

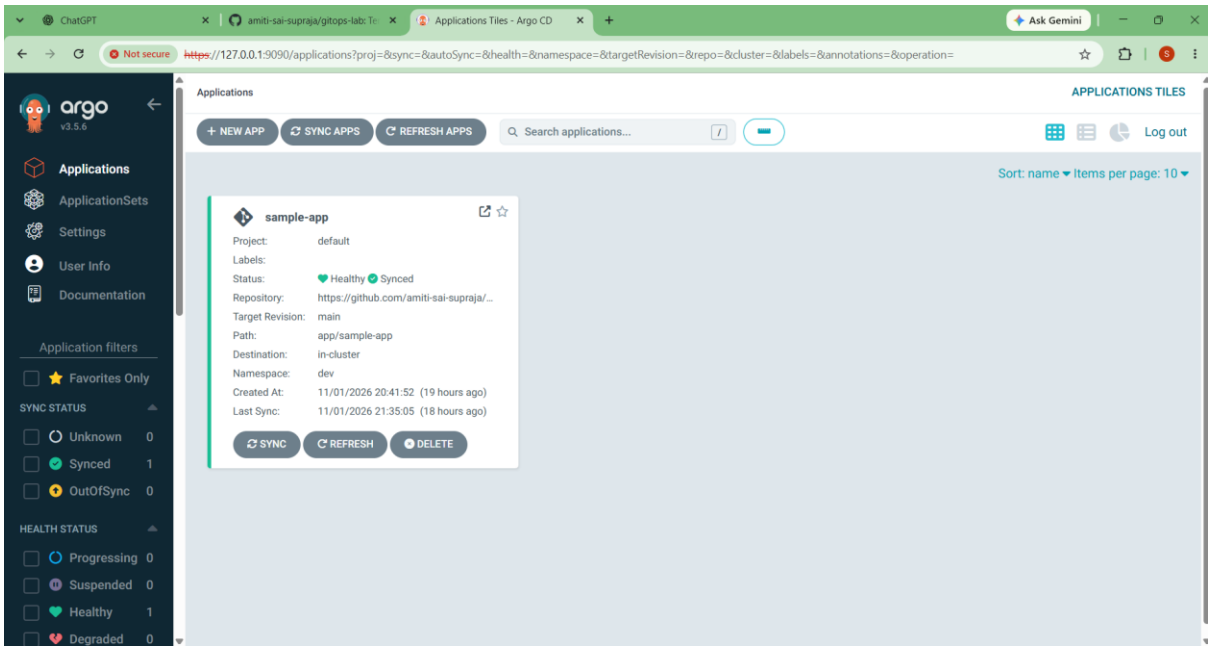
GitHub Repository:

github.com/amiti-sai-supraja/gitops-lab



I stored the Helm application and GitOps configuration in GitHub

ArgoCD

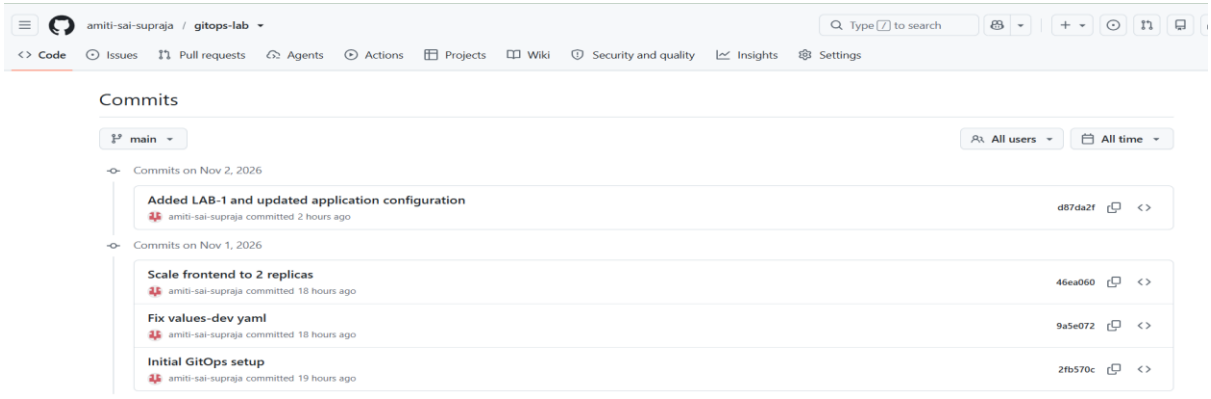


ArgoCD successfully synchronized the application from GitHub into the Kubernetes cluster.

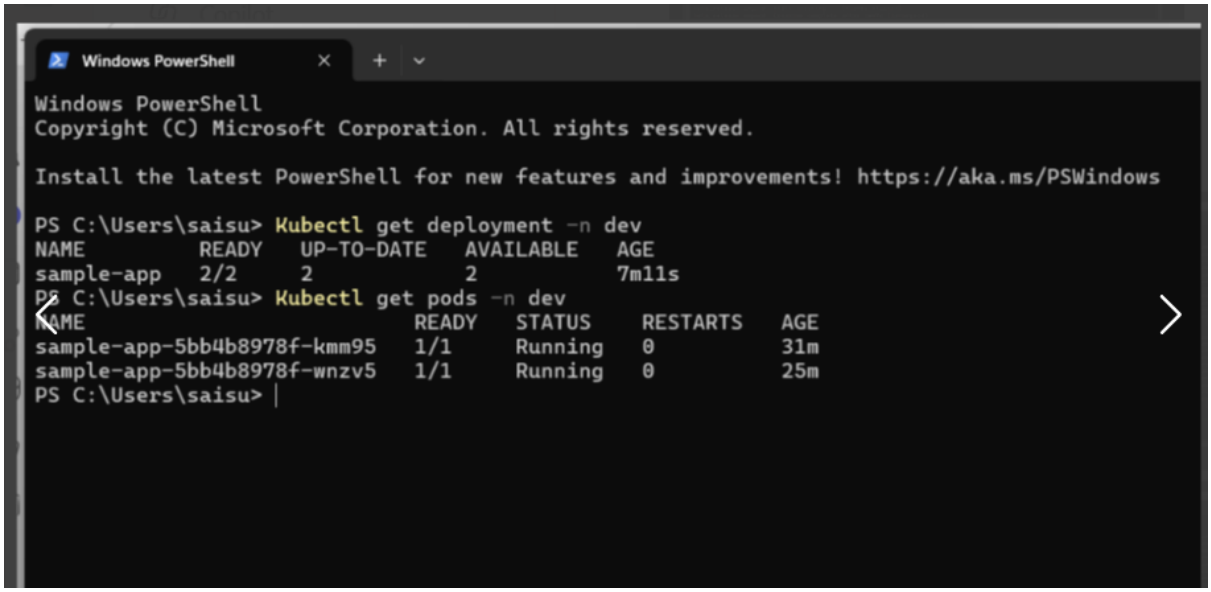
Replica Count

A replica is a running copy of an application. One replica means one pod. Two replicas mean two pods providing higher availability.

GitOps Deployment

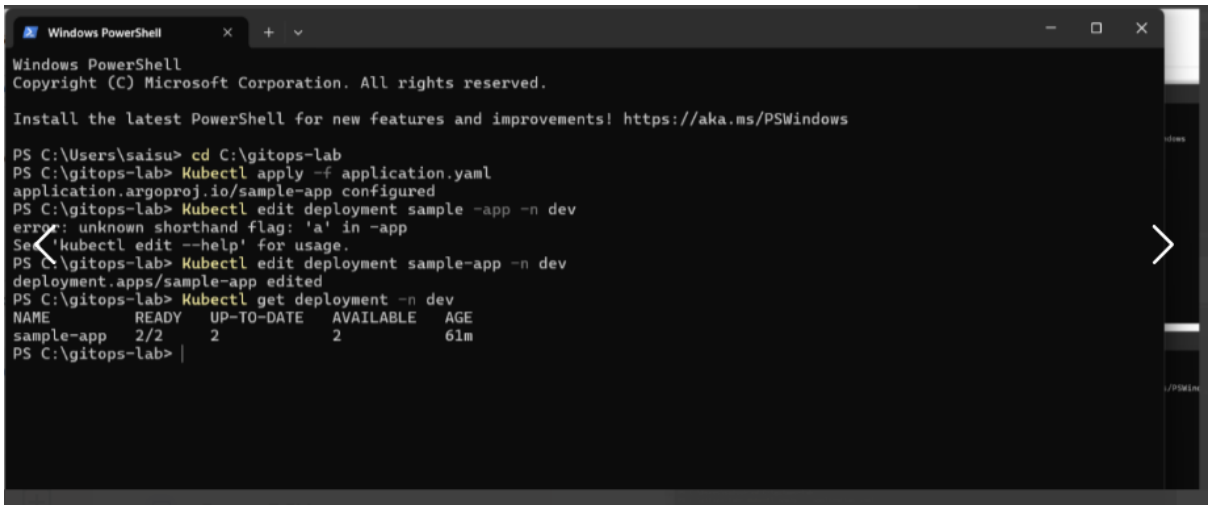


I changed replicaCount from 1 to 2 in values-dev.yaml and pushed the change to GitHub.



ArgoCD automatically detected the Git commit and created a second pod without me running kubectl apply.

Self-Healing Demonstration



I manually modified the deployment using kubectl edit. Since self-heal was enabled, ArgoCD detected the drift and automatically restored the deployment back to the Git version.

LAB-1.md

GitOps is safer because Git acts as the single source of truth. Every change is recorded and reviewed through commits. ArgoCD continuously compares the cluster state with Git and automatically corrects configuration drift. This reduces human error and ensures consistent deployments.

I Pushed Everything into my GitHub account .

https://github.com/amiti-sai-supraja/gitops-lab